InferenceChain

A Decentralized Blockchain Native to AI Inference

Quality of Inference Consensus · Proof of Quality of Inference · Sharded BFT

Technical Whitepaper & Vision Document

Research Preview · v0.5 · April 2026

Dimitrios Papaioannou

Aristotle University of Thessaloniki (AUTH) · InferenceChain Research

Table of Contents

---

1. Abstract

InferenceChain is a novel blockchain protocol designed specifically for decentralised AI inference. Unlike general-purpose blockchains that treat computation as an opaque black box, InferenceChain introduces inference quality as a first-class consensus property. Validators do not merely agree on transaction order — they agree on the correctness of deep neural network (DNN) outputs.

Three novel primitives drive the protocol: **Quality of Inference Consensus (QoI)**, a PBFT-derived algorithm that uses cosine similarity between inference vectors to reach Byzantine-fault-tolerant agreement; **Proof of Quality of Inference (PoQI)**, an on-chain reputation system that quantifies each

validator's historical accuracy and weights their future influence accordingly; and **Sharded BFT (S-BFT)**, a horizontal scaling layer that partitions the validator set into independent committees to achieve linear throughput growth without sacrificing BFT safety guarantees.

A working testnet implementation — including the full consensus engine, P2P gossip layer, on-chain state machine, model admission protocol, OOD-based quality assessment, reliability tracking with anti-gaming mechanisms, and a live web dashboard — has been developed and is described in this document.

2. Problem Statement

The rise of AI inference as a service has created a critical centralisation risk. Today, virtually all AI inference is performed by a handful of large cloud providers. Users have no way to verify that the model they requested actually ran, that the output has not been tampered with, or that the service provider is not selectively serving degraded results.

Existing blockchain-based approaches fall into two categories, both inadequate:

Approach	Limitation
Optimistic execution	Assume the inference result is correct and only dispute on-chain after the fact. Disputes are slow, expensive, and rely on a trusted arbiter.
Commit-reveal schemes	A node commits to a result hash then reveals it. This prevents copying but provides no quality guarantee — a node can commit a random hash.
Reputation without verification	Systems that track reputation without on-chain verifiable proof of model quality are gameable. A node can report high accuracy it never achieved.
General-purpose BFT	Standard PBFT agrees on values, not on quality. Two nodes producing different (but equally valid) inference outputs would cause consensus to fail even if both are correct.

InferenceChain addresses all four limitations simultaneously. Consensus is reached on inference quality, verified by mathematical comparison (cosine similarity). Reputation is an on-chain quantity, updated deterministically by protocol rules. Admission requires cryptographic proof of model capability before a node may participate. In-distribution awareness (OOD scoring) and hidden challenges prevent gaming and reward domain expertise.

3. Architecture Overview

InferenceChain separates concerns cleanly across four layers:

Layer	Responsibility
Network Layer	P2P gossip over WebSocket connections. Each node maintains persistent connections to peers. Messages are typed (CONSENSUS, TRANSACTION, BLOCK, PEER_DISCOVERY) and routed accordingly.
Consensus Layer	Two concurrent consensus protocols. QoI consensus runs when an INFERENCE_REQUEST transaction enters the mempool. PoS consensus runs for all other (simple) transaction blocks.
State Layer	A single ChainState object is the authoritative source of truth — balances, stakes, reputations, registered nodes, inference log, reliability scores, challenge records. All reads bypass any off-chain registry.
Application Layer	FastAPI REST + WebSocket API served on port 8000. React dashboard at `/ui`. Desktop sidecar (PyInstaller) with OOD profiling on port 47291. All data read directly from chain state.

4. Quality of Inference Consensus (QoI)

QoI is a three-phase Byzantine-fault-tolerant consensus protocol derived from PBFT, adapted to agree not on an arbitrary value but on the quality of a DNN inference result. The key insight is that correct DNN inference on the same image produces output probability vectors that are mathematically close — measurable by cosine similarity.

The protocol requires $n \geq 3f + 1$ validators to tolerate f Byzantine nodes.

Phase 1 — PRE-PREPARE

The primary (proposer) receives an INFERENCE_REQUEST containing an image hash. It fetches the image from the requesting client, runs it through its local DNN model, and broadcasts a PRE_PREPARE message containing its output probability vector (softmax over 10 CIFAR-10 classes).

Phase 2 — PREPARE

Each DNN validator independently runs the same image through its own local model. It computes the cosine similarity between its output vector and the primary's vector. If the similarity exceeds the threshold, it broadcasts a PREPARE message with its own vector. The primary collects PREPARE messages until it has 2f+1.

Phase 3 — COMMIT

Once 2f+1 PREPARE messages are collected, the primary broadcasts a COMMIT message. Each validator that has seen 2f+1 PREPAREs broadcasts its own COMMIT. When a node collects 2f+1

COMMITs, the round is finalised. A QoI block is appended to the chain containing the consensus result, all signatures, and reputation deltas.

Quality Scoring

After each round, each validator's output vector is compared pairwise against all others. Validators whose vectors lie within the cosine similarity threshold of the majority cluster are marked as honest and receive reputation increases proportional to their similarity score. Outliers (Byzantine or low-quality nodes) are penalised.

---

5. Proof of Quality of Inference (PoQI)

PoQI is the on-chain reputation system that tracks each validator's cumulative inference quality. It is not a separate consensus protocol — it is a deterministic state transition that occurs at the end of every QoI round.

Reputation Parameters

Parameter	Value
Initial reputation	0.0 (all nodes start equal)
Max gain per round	0.5 (REP_CAP_PER_ROUND)
Penalty per Byzantine act	0.3 (REP_PENALTY)
Leader bonus per round	+0.1 on top of quality-proportional gain
Reputation cap	No hard cap — grows unboundedly with consistent quality
Reputation floor	0.0 (cannot go negative)

Reputation serves two functions: it weights the proposer election (higher reputation → higher probability of being selected as primary), and it weights block reward distribution. A validator that consistently produces high-quality inference results earns disproportionately more INFER tokens over time — creating a strong economic incentive for model quality maintenance.

---

6. Proof of Model (PoM) — Admission Protocol

Before a node may participate in QoI consensus, it must pass the Proof of Model admission protocol. PoM prevents Sybil attacks and ensures that every active DNN validator in the network actually possesses a model that meets minimum quality standards.

Admission Flow

Step	Action
Step 1 — Register	Node submits a NODE_REGISTER_DNN transaction containing: model name, weights SHA-256 hash (on-chain commitment), endpoint URL, dataset ID, architecture JSON, and public key. Node status: PENDING_POM.
Step 2 — Challenge	Protocol generates a MODEL_CHALLENGE transaction with 50 deterministically selected CIFAR-10 test samples. Seed = SHA-256(prev_block_hash + node_address) — fully reproducible by any verifier.
Step 3 — Response	Challenged node runs the 50 samples through its model and broadcasts a MODEL_RESPONSE transaction with its predictions within the timeout window (30 seconds).
Step 4 — Verification	Existing DNN validators independently verify the response against known CIFAR-10 test labels. Each broadcasts a MODEL_VERIFY transaction with pass/fail.
Step 5 — Admission	If ≥ 80% accuracy is achieved and enough verifiers agree, a NODE_ADMITTED transaction is committed. Node becomes active. If it fails: NODE_REJECTED — node must re-register.

The weights hash committed in Step 1 binds the node to a specific model. If a node later attempts to serve a different model, the hash mismatch is detectable on-chain by any verifier — providing cryptographic proof of model substitution fraud.

---

7. Sharded BFT Architecture (S-BFT)

Standard PBFT has $O(n^2)$ message complexity — as the validator set grows, the communication overhead grows quadratically, making it impractical beyond ~100 nodes. S-BFT solves this by electing a small quorum of K nodes per inference request, rather than involving the full validator set.

The Scaling Problem

With N DNN nodes in the network, running QoI across all of them for every inference request is unscalable:

- $O(N^2)$ message complexity
- Latency that tracks the slowest node
- Most nodes having no familiarity with the requested model or dataset

S-BFT reduces the active committee to K = 10 nodes (configurable), achieving $O(K^2)$ message complexity independent of total network size.

Per-Request Quorum Selection

For each inference request, a quorum is elected as follows:

1. **Filter** eligible DNN validators by model name and dataset ID matching the request
2. **Compute seed** = SHA-256(block_hash || request_id) — unique per round, reproducible by any node
3. **Sample K nodes** from the filtered pool using the seed — without replacement, deterministically
4. **Sort** selected nodes by node_id for canonical ordering

Quorum assignment is verifiable by any network participant and unpredictable before the inference request arrives — preventing adversaries from pre-positioning nodes.

Quorum Parameters

Parameter	Value
Target quorum size	10 nodes (ideal)
Floor	4 nodes minimum — gives f=1 (requires 3f+1)
Ceiling	19 nodes maximum — gives f=6
Fallback	If filtered pool < 4 nodes, all eligible DNN validators participate

Local BFT Parameters

Each quorum operates with its own fault-tolerance parameters, independent of the global chain f:

$$f = \lfloor (K - 1) / 3 \rfloor$$

$$\text{threshold} = 2f + 1$$

For K=10: f=3, threshold=7. For K=4: f=1, threshold=3.

Nodes not elected to the quorum stand by for one block slot — they do not participate and do not interfere. The committed QoI block arrives via P2P gossip and is ingested normally by all nodes.

Quorum Integrity at Commit

When a QoI block is sealed, the protocol validates that every commit signature originated from a node in the elected quorum. Signatures from outside the quorum are stripped — those nodes receive no reward. The elected quorum node IDs are embedded in the ConsensusOutcome and written to chain in the CONSENSUS_RESULT transaction.

---

8. Block Structure & Transaction Types

Two Block Types

Block Type	Description
QoI Block	Produced after a successful QoI consensus round. Contains exactly one INFERENCE_REQUEST, plus system transactions: INFERENCE_RESPONSE, CONSENSUS_RESULT, REWARD entries for each participating validator, and reputation delta updates. Committed by the QoI state machine.
PoS Block	Produced for all simple (non-inference) transactions. Contains up to 50 simple transactions per block. Proposer is selected by stake × reputation weighting. Committed by the PoS consensus machine with 2f+1 votes.

Transaction Taxonomy

Type	Family	Purpose
TOKEN_TRANSFER	Simple	Peer-to-peer INFER token transfer
STAKE / UNSTAKE	Simple	Lock / unlock tokens as validator stake
NODE_REGISTER_DNN	Simple	Register as a DNN validator (triggers PoM)
NODE_REGISTER_POS	Simple	Register as a PoS-only validator
MODEL_RESPONSE	Simple	Node's answer to a PoM challenge
INFERENCE_REQUEST	Inference	Client submits an image for consensus inference
INFERENCE_RESPONSE	System	Protocol records a validator's output vector
CONSENSUS_RESULT	System	Final agreed inference result
REWARD	System	Token minted to a validator for honest participation
SLASH	System	Tokens burned from a validator for Byzantine behaviour
MODEL_CHALLENGE	System	Protocol challenges a pending node's model
MODEL_VERIFY	System	Existing validator verifies a challenge response
NODE_ADMITTED	System	Node passed PoM — becomes active
NODE_REJECTED	System	Node failed PoM — must re-register

9. OOD-Aware Quality & Anti-Gaming Infrastructure (NEW)

9.1 Out-of-Distribution Scoring

Every admitted DNN node automatically calibrates an **Out-of-Distribution (OOD) scorer** the moment its `NODE_ADMITTED` transaction is committed. The scorer measures how *in-distribution* (familiar) a candidate image is for a given node's model.

Scoring Method: Hybrid Mahalanobis + Energy

- **Shared Encoder**: All nodes use frozen ViT-B/16 (torchvision.models.vit_b_16)
- **Class-Conditional Profiles**: Mahalanobis distance on penultimate-layer features
- **Knowledge Score K(x)**: Logistic fusion yielding $K(x) \in [0, 1]$
- **Query-Time Signing**: Optional Ed25519 signatures for authenticity

9.2 Multi-Factor Weighted Quorum Selection

The quorum incorporates three factors: **stake**, **knowledge**, and **reliability**.

Weighting Formula

$$w_i = \text{stake}_i \times K_i^{\gamma} \times R_i^{\beta}$$

where $\gamma = 1.5$ and $\beta = 1.0$ (tunable).

Deterministic Weighted Sampling

Using Efraimidis-Spirakis algorithm for fair, reproducible selection without replacement.

9.3 Node Reliability Tracking (Anti-Gaming)

Per-node **EMA-based reliability** with dual update pathways:

- **Regular QoI rounds** (α=0.2): Target = 0.6 + 0.6×QoI_score (honest) or 0.2 (Byzantine)
- **Challenge rounds** (α=0.45): Target = 0.65 + 0.55×K (honest) or 0.25 – 0.2×K (Byzantine, knowledge-weighted)

Reliability score $\in$ [0.05, 1.5], clamped to prevent extremes.

Per-node counters: rounds, honest_rounds, byzantine_rounds, challenge_rounds, challenge_penalties.

9.4 Deterministic Hidden Challenge Rounds

Challenge Selection at configurable rate (default 15%):

$$\text{is\_challenge} = \frac{\text{int}(SHA256(\text{request\_id} | \text{block\_hash})[:4])}{2^{32}} < \text{rate}$$

Challenge Records: Timestamp, request_id, image_hash, block_hash, per-node honesty, OOD/knowledge scores.

Challenge-Driven Updates: Knowledge-weighted reliability adjustments — high-knowledge dishonest nodes receive strong penalties.

---

10. What Is Already Built

The following components are fully implemented and operational on the local testnet as of April 2026.

✓ Blockchain core with QoI & PoS blocks, Merkle roots, persistent SQLite storage

✓ ChainState with balances, stakes, reputations, registered nodes, reliability/challenge records

✓ S-BFT quorum selection with multi-factor weighting (stake × knowledge × reliability)

✓ QoI consensus engine (3-phase PBFT, view-change, timeout)

✓ PoS consensus for simple transactions

✓ Proof of Model admission protocol

✓ OOD profiling with shared ViT-B/16 encoder and Mahalanobis profiles

✓ Reliability tracking with dual EMA pathways

✓ Deterministic hidden challenges with challenge records

✓ Wallet & signature verification (secp256k1, BIP39)

✓ P2P gossip layer with WebSocket connections

✓ REST API (40+ endpoints) with full OpenAPI documentation

✓ Model registry (7 CIFAR-10 architectures)

✓ Frontend dashboard (React)

✓ Desktop application (PyInstaller + Electron)

✓ Test suite (37/37 QoI consensus tests passing)

---

11. Tokenomics — INFER Token

The INFER token is the native currency. It serves three roles: **economic incentive** for inference quality, **security bond** making Byzantine behaviour costly, and **governance weight** for future votes.

Supply Parameters

Parameter	Value

Token name	INFER
Maximum supply	100,000,000 INFER
Genesis allocation	10,000,000 INFER (10%)
Block reward	10.0 INFER per QoI round
Leader bonus	+2.0 INFER
Slash penalty	100.0 INFER
Min DNN stake	1,000 INFER
Min PoS stake	500 INFER

Reward Distribution

Recipient	Amount
Primary	Base reward × (1 + leader_bonus_ratio)
Honest validators	Proportional share weighted by cosine similarity
Byzantine validators	Zero reward + 100 INFER slashed
PoS validators	Proportional share (separate pool)

Stake × Reputation Interaction

$$\text{reward\_share}(i) = \frac{\text{stake}(i) \times \text{reputation}(i)}{\sum_j (\text{stake}(j) \times \text{reputation}(j))}$$

Quality and capital are equally weighted — new nodes earn nothing until demonstrating quality, preventing pure capital dominance.

Emission Schedule

~6.3M INFER/year maximum theoretical rate at 10 INFER per 5-second block target. Remaining 90M supply: ~14 years to emit. Actual emission demand-driven.

---

12. Vision & Roadmap

InferenceChain's vision: become the trust layer for AI inference — the protocol any application can use to prove that a specific AI model produced a specific output, verified and economically incentivised.

Phase 1 — Foundation (Current: v0.5)

✓ Core consensus, reputation, model admission

✓ Sharded BFT with multi-factor weighting

✓ OOD-aware scoring and reliability tracking

✓ Deterministic hidden challenges

✓ Full implementation and testnet

Phase 2 — Scaling & Security (Q3 2026)

■ Tag-driven challenge pools (domain taxonomy)

■ Output-space label canonicalization

■ Signature aggregation (BLS)

■ Peer discovery (Kademlia DHT)

■ Multi-model support

■ Light client protocol

Phase 3 — Ecosystem (Q1 2027)

■ Smart contracts (Turing-complete VM)

■ Cross-chain bridge (EVM)

■ Model marketplace

■ DAO governance

■ Mainnet launch

■ Python/JS SDK

The Broader Vision

As AI becomes consequential across healthcare, law, finance, autonomous systems — verifiable, auditable AI becomes necessity. InferenceChain provides cryptographic infrastructure for this audit trail: any inference result on-chain can be independently re-verified by anyone, at any time, with mathematical certainty.

End state: AI inference as transparent, economically incentivised, cryptographically auditable process — not a black box.

---

13. Conclusion

InferenceChain introduces a fundamentally new class of blockchain — designed from first principles for AI inference. The three core innovations (QoI, PoQI, S-BFT) with OOD-aware weighting, reliability tracking, and hidden challenges solve trustless, verifiable, quality-assured DNN inference at scale.

Complete working implementation demonstrates protocol is not merely theoretical. Testnet runs QoI rounds, admits validators through PoM, tracks on-chain reputation and reliability, manages challenge

rounds, and serves full-featured dashboard — all operating coherently.

INFER token creates self-sustaining economic system where quality and security incentives align: stake securing network proportionally determines reward shares; reputation — earned only through quality — amplifies stake's influence.

InferenceChain is positioned to become the trust infrastructure for the AI era — making AI outputs as verifiable as blockchain transactions.

---

14. Appendix A — Wallet Cryptography & Transaction Construction

This appendix documents the wallet implementation currently shipped in the React frontend and how it maps to backend signature verification.

14.1 Key Material, Address Scheme, and Storage

The frontend wallet (`frontend/src/utils/wallet.js`) implements a deterministic secp256k1 flow:

Artifact	Implementation Detail
Mnemonic	12-word BIP39 (128-bit entropy)
Seed derivation	`bip39.mnemonicToSeedSync`
Private key	First 32 bytes of seed (SHA-256 fallback only for invalid all-zero edge case)
Public key	Uncompressed secp256k1, 64-byte payload (without `0x04` prefix)
Address	First 20 bytes of SHA-256(public_key_bytes), hex encoded

Private keys are never persisted in plaintext. Persisted wallet records contain only:

- `address`
- `publicKey`
- `encrypted = { ciphertext, salt, iv }`

14.2 At-Rest Encryption Model

Wallet encryption is AES-256-GCM with PBKDF2-SHA256 key derivation:

Parameter	Value
KDF	PBKDF2
Hash	SHA-256

Iterations	100,000
Salt length	16 bytes
IV length	12 bytes
Cipher	AES-GCM-256

This model provides integrity protection (GCM auth tag) and confidentiality for local storage snapshots.

14.3 Canonical Transaction Signing

Transactions are signed over deterministic canonical JSON:

1. Serialize payload with sorted keys (`canonicalJson`).
2. Compute `tx_id = SHA256(canonical_json)`.
3. Hash tx_id bytes again to match Python backend verification expectation.
4. Sign with secp256k1 compact signature.

Supported client-side transaction builders:

- `token_transfer`
- `stake`
- `unstake`

14.4 Wallet UX Security Controls

Implemented controls in current frontend:

- Password strength scoring (length + class diversity checks)
- Minimum password acceptance policy
- Locked-by-default behavior with explicit unlock step
- Local-only key management (no key upload path in normal flow)

---

15. Appendix B — Desktop Application Architecture (Electron + Sidecar)

The desktop distribution is a two-process architecture:

Process	Runtime	Responsibility
Shell	Electron main/renderer	Native app UX, filesystem dialogs, settings, local orchestration
Sidecar	FastAPI + Python	Validation, training, OOD profile fitting/scoring, chain submission

15.1 Electron Main Process Responsibilities

`desktop/electron/main.js` currently provides:

- Sidecar spawn and health checks (`/health` on port `47291`)
- User data and settings persistence (`closeBehavior`, `startToTray`)
- Tray-mode lifecycle (background mode vs quit)
- Hardened renderer boundary via IPC handlers

IPC handlers include:

- `sidecar:port`
- `dialog:openFile`
- `fs:readFile`
- `app:dataPath`
- `settings:get`, `settings:set`

15.2 Sidecar API Modules

`desktop/sidecar/main.py` mounts routers:

Prefix	Module	Function
`/validate`	`routes/validate.py`	Architecture and model validation
`/train`	`routes/train.py`	Train job setup, SSE metric streaming, stop/status
`/chain`	`routes/chain.py`	Manifest signing, submission, sync status, peer mgmt
`/ood`	`routes/ood.py`	OOD profile fitting and per-image familiarity scoring

15.3 Training Lifecycle in Desktop Flow

The current train flow is single-run-at-a-time:

1. `POST /train/start` stores config.
2. `GET /train/stream` emits SSE events per epoch.
3. `POST /train/stop` requests cancellation.
4. `GET /train/status` reports active/inactive state.

This design keeps orchestration simple and deterministic for first-generation validator onboarding.

15.4 Manifest Signing and Deferred Submission

`/chain/sign-checkpoint` generates:

- `checkpoint_hash`
- architecture hash
- dataset hash
- final metrics
- optional OOD profile payload

Manifest is signed with Ed25519 and submitted via `/chain/submit`. If chain is unreachable, manifests are persisted to `pending_submissions` for delayed retry.

---

16. Appendix C — LLM Orchestration Layer (Route/Analyse/Decompose/Explain)

The LLM layer is optional and explicitly non-consensus-critical.

16.1 Safety Boundary

`core/api/routes/llm.py` enforces a strict boundary:

- If no orchestrator is configured: returns HTTP `503` with setup instructions.
- Consensus behavior remains unchanged.
- QoI, PoQI, PoM, and block validation continue without LLM dependencies.

16.2 Endpoint Contracts

Endpoint	Intent	Output
`POST /llm/route`	Recommend `(model_hint, dataset_id)` for task description	Route decision + confidence
`POST /llm/analyse`	Detect anomalous validators from chain summary	Suspicious node list + re-challenge suggestions
`POST /llm/decompose`	Split complex jobs into parallel subtasks	Structured subtask list
`POST /llm/explain`	NL Q&A; over current state	Text answer

16.3 Provider Abstraction

The orchestrator supports provider pluggability:

- Ollama local inference
- OpenAI cloud inference
- Mock deterministic provider for tests and offline environments

This enables deterministic CI while preserving real-world deployment options.

17. Appendix D — End-to-End Transaction Lifecycle & State Transitions

This appendix formalizes practical transaction movement from client to committed state.

17.1 Flow for Simple Transactions

1. Wallet builds and signs tx (`token_transfer`, `stake`, `unstake`).
2. Node REST endpoint validates signature and schema.
3. Tx enters mempool.
4. PoS proposer includes tx in PoS block.
5. Block commits after threshold votes.
6. ChainState mutates balances/stakes/nonces.

17.2 Flow for Inference Transactions

1. Client submits `INFERENCE_REQUEST`.
2. S-BFT elects quorum from eligible DNN validators.
3. QoI phases run: PRE-PREPARE -> PREPARE -> COMMIT.
4. Cosine-similarity agreement determines accepted cluster.
5. Protocol emits `CONSENSUS_RESULT`, rewards, and penalties.
6. Reputation and reliability updates are applied deterministically.

17.3 Deterministic State Discipline

All economic and consensus-relevant fields are driven through on-chain state transitions; no hidden side databases are trusted for settlement decisions.

18. Appendix E — API Surface & Endpoint Contracts

InferenceChain exposes a multi-domain API through FastAPI (chain, state, tx, inference, p2p, dashboard, websocket, llm).

18.1 Core API Domain Groups

Domain	Typical Paths	Purpose

Chain	`/chain/*`	Height, blocks, canonical history
State	`/state/*`	Balances, reputations, validator views
Transactions	`/tx/*`	Submit and inspect transactions
Inference	`/inference/*`	Inference request and result retrieval
P2P	`/p2p/*`	Peer status and peering operations
Dashboard	`/dashboard/*`	UI-optimized aggregates
WebSocket	`/ws/*`	Real-time event streaming
LLM	`/llm/*`	Optional orchestration layer

18.2 Desktop Sidecar API

Local sidecar endpoints intentionally run on loopback (`127.0.0.1`) to reduce exposure surface.

18.3 Contract Evolution Policy (Current)

The active codebase favors backward compatibility at route level where practical, but does not yet implement a fully versioned `/v1`, `/v2` namespace contract. This is a known future hardening item.

---

19. Appendix F — Data, Models, and Training Pipeline

19.1 Model Zoo and Baselines

Current repository includes CIFAR-family architectures under `models/architectures` (ResNet variants, WideResNet, VGG, DenseNet, MobileNetV2, PyramidNet).

19.2 Dataset and Training Inputs

Desktop flow supports dataset references and archives, with training configuration captured in the signed manifest to provide reproducibility metadata.

19.3 OOD Profile Generation

`desktop/sidecar/training/ood_profile.py` implements the hybrid familiarity method:

- shared frozen encoder (ViT-B/16)
- class-conditional feature statistics
- optional energy statistics
- fused knowledge score for quorum weighting and reliability dynamics

19.4 Reproducibility Artifacts

Every admitted model path can be traced via:

- architecture hash
- dataset hash
- checkpoint hash
- training config
- final metrics
- optional OOD profile hash

---

20. Appendix G — Security Model & Threat Analysis

20.1 Threat Classes Addressed

Threat	Mitigation in Current System
Sybil validator admission	PoM challenge-response + threshold verification
Model substitution after admission	On-chain model hash commitment
Low-quality random outputs	QoI cosine clustering + penalties
Strategic sandbagging	Reliability score + hidden challenge rounds
Quorum manipulation	Deterministic seeded weighted sampling
Private key theft at rest (client)	AES-GCM encrypted wallet store

20.2 Remaining Risks (Open)

- Local endpoint misconfiguration in operator deployments
- Dataset poisoning if operator training data is malicious
- Side-channel leakage outside protocol boundary (host compromise)
- Adversarial inputs beyond current static threshold assumptions

These are tracked roadmap areas, not ignored risks.

---

21. Appendix H — Test Coverage, Validation, and Known Gaps

21.1 What Is Covered

The repository contains unit/integration test suites under `tests/`, including consensus and blockchain primitives, with QoI-focused validation scenarios actively maintained.

21.2 Practical Validation Loops

Project validation currently uses:

- Python unit tests (`pytest`)
- frontend production builds (`react-scripts build`)
- desktop packaging pipeline (PyInstaller + Electron)
- deterministic API smoke tests in local testnet mode

21.3 Known Gaps

- More adversarial simulation matrices for large validator counts
- API schema versioning and compatibility gates
- Expanded fuzzing for transaction payload edge cases
- Additional long-horizon economic simulation for token dynamics

---

22. Appendix I — Deployment, Operations, and Observability

22.1 Runtime Topologies

Supported current operation modes:

- Pure Python node runtime
- Multi-node local testnet
- Frontend dashboard deployment (Vercel/static)
- Desktop app distribution (portable executable and release assets)

22.2 Operational Telemetry

Operators can inspect:

- chain height and finality progression
- peer connectivity
- validator reliability trends
- challenge history and penalties
- training lifecycle and pending submissions

22.3 Release Discipline

Desktop release workflow is tag-driven (`v*`) and publishes assets to GitHub Releases via CI. Website Download App links should target `releases/latest` so current binaries are always discoverable.

22.4 Current-State Summary

InferenceChain, in its April 2026 state, is already a complete end-to-end prototype stack: wallet -> transaction signing -> consensus -> reputation/reliability -> dashboard + desktop operator flow -> optional LLM coordination. The remaining roadmap focuses on scale, hardening, and ecosystem expansion rather than foundational feasibility.

Dimitrios Papaioannou

InferenceChain Research · Aristotle University of Thessaloniki (AUTH)

April 2026 · Research Preview v0.5